OBDH Model Software Development and Operations

R7023E On-Board Data Handling – Project Report

Oliver Witzel, Isaac Edwards, Petter Bäckman, Anton Damberg

October 31, 2026

1 Introduction

This report covers the Ground (Gnd), Platform OBC (PF) and Payload ICU (PL) software developed for the R7023E OBDH assignment. The three programs are written in Python and run as two Docker containers that communicate over TCP. The design is described in the OBSW Software Design Document (SDD), and the code is available at `https://github.com/Oliver-Witzel/R7023R-onboard-docker`. We were initially planning on using three dockers, this was later changed since we considered calling a class is identical as if we used a third docker. This is a preface to why there might be contradictions in the SDD and function tree documents. Second, the planned JSON messages were replaced by CCSDS Space Packets using the `spacepackets` library. Where the function tree describes the original three-container design, it should be read as the initial design.

Requirements R01–R32 were derived from the assignment text and numbered by the team; they are the same as in the SDD traceability matrix.

2 Version control

We distributed the work by dividing the project into different parts. Having every member be a part of all the code is impossible since writing code as four doesn't make any sense. We split it up into the different elements (ground / obc / payload). The code version was maintained using github were we pushed and pulled directly into main. The function tree and SDD were made in group and the final report was made through discussion.

- Tool: git, hosted on GitHub.
- Branching: one branch per element (ground / obc / payload), merged into main after testing.
- Coordination: the message format and the MIB were agreed first and kept in the repository so all three programs used the same version.

3 Verification matrix

Table 1 lists every requirement, how it was tested, and the result. Tests were run on 31:th of October with ground and obc alonside PL running. Summary: 26 of 32 requirements passed

were 3 of them were minor errors and the last 3 were not implemeted.

Table 1: Verification matrix

Req	Requirement	Test performed	Result
R01	Three separate SW programs	[test]	Partial
R02	Text prompt for TCs	SET-OBT	Pass
R03	TM displayed as received	SET-OBT	Pass
R04	HK reception timestamp + logfile	REQ-HK	Pass
R05	MIB as text file	Start up	Pass
R06	SAFE: PL cmds rejected, PL OFF	PL-TAKE-IMAGE w/o PL ON	Pass
R07	At least one non-SAFE mode	SET-OBC-MODE TARGET	Pass
R08	PL commandable only in non-SAFE	PL-TAKE-IMAGE w MODE=SAFE	Pass
R09	SAFE implies PL shutdown	SET-OBC-MODE SAFE	Pass
R10	Mode transitions take time	SET-OBC-MODE TARGET	Pass
R11	Mode acquisition event TM	SET-OBC-MODE TARGET	Pass
R12	Invalid commands rejected with reply	NOT-A-COMMAND	Pass
R13	Accepted commands acknowledged	SET-OBT	Pass
R14	Time-tagged TCs	Not tested	Not tested
R15	Overdue time tag rejected	Not tested	Not tested
R16	Time-tagged TC executed when due	Not tested	Not tested
R17	Common time base Gnd / PF	SET-OBT	Pass
R18	PL OFF / ON simulated	SET-OBC-MODE TARGET -> SET-PL-POWER 1	Pass
R19	PL reports image start and finish	SET-OBC-MODE TARGET -> SET-PL-POWER 1 -> PL-TAKE-IMAGE	Pass
R20	PL image info to Gnd in HK-PL	SET-OBC-MODE TARGET -> SET-PL-POWER 1 -> PL-TAKE-IMAGE	Pass
R21	PL ON/OFF event TM	SET-OBC-MODE TARGET -> SET-PL-POWER 1	Pass
R22	Command target identification	SET-OBT	Pass
R23	TC and TM sequence counts	SET-OBT	Partial
R24	Cyclic HK vs. event TM	REQ-HK	Pass
R25	SCET until OBT set	SET-OBT	Pass
R26	SET-OBT TC	SET-OBT	Pass
R27	hh:mm:ss in TM and time tags	SET-OBT	Partial
R28	OBT periodically in HK	startup	Pass
R29	Event TM carries OBT	SET-OBT	Pass
R30	PL image log with take time	SET-OBC-MODE TARGET -> SET-PL-POWER 1 -> PL-TAKE-IMAGE	Pass
R31	PL knows actual OBT	SET-OBT	Pass
R32	Cyclic loops, PF polls uplink	startup	Pass

4 Requirements not tested or not met

- **R01**: Ground and OBC runs as separate programs, the payload is a separate module inside the OBC process.
- **R14-16,R27**: We didn't implement the time-tags.
- **R23**: Seq timeout has a bug that some ACK and NACK aren't registered in the seq timeout queue which results in acknowledged TC:s being false logged as timed-out even though the acknowledgment has been received.

5 Lessons learned

- **What worked:** We created the data_structures.txt file alongside the MiB, FT and SDD, the aim of the file was to have a short list of all TC/TMs and their arguments. They where all written in the format that they would be parsed into as a bit array. This document as well as the function tree and SDD allowed us to work on the three systems independently of each other, which allowed for a more equal distribution of work.Moreover, the docker setup, although clunky to setup, made both development and communication between programs easier. There was no mismatch between versions and package managers, since docker builds a complete image with the required packages.
- **What we would do differently:** Perhaps we should have used something like JSON instead of parsing and framing our own packets, but it was a interesting side tangent. Furthermore, we could have managed our time better to allow for a more complete set of programs. Another thing that could have further improved efficiency would have been to create a branch each for ground, obc and payload, this would have minimized the constant merge conflicts on main.Lastly the docker containers although useful presented yet another learning step for those of us that where unfamiliar with docker, so one could argue that ti would perhaps been easier if we choose a more simple setup.
- **How the prototype could be improved:** A clear improvement would be to actually implement timetagging. But on a more grander view it would have been interesting to run the containers on separate machines, perhaps one at home and one on campus. This is one of the features of using docker and a setup like this would allow for a more accurate up/downlink scenario since the physical distance would induce noise, lag and even potential packet loss.